

Digital Assistant (Agentic AI)

Production-style serverless AI assistant with tools, memory, async workers, and quota governance.

Live App: <https://digital-assistant.agentairg.site>

Deployment: Terraform + GitHub Actions

Architecture: AWS Serverless

Problem

- Most chat demos break in production: timeout failures, unreliable tool outputs, and missing usage controls.
- Long-running tasks exceed synchronous API limits and degrade UX.
- Without verifiable actions and observability, operations and trust become fragile.

Solution & Core Features

- Tool-enabled agent: web search, PDF export, email, file upload parsing, memory extraction.
- Async execution: POST /chat returns job_id, worker completes long tool chains.
- Daily quota enforcement: tokens/PDF/email tracked server-side with deterministic limits.
- Human-approved memory: candidate memories require explicit approval before reuse.

- Reliability guardrails: tool-claim validation and safe fallbacks for failed tool actions.
- Upload path optimized for production: browser direct-to-S3 via presigned URLs.
- Responsive UI tuning for mobile/tablet chat header actions and quota visibility.
- Split domains in prod: website on CloudFront domain, API on API Gateway custom domain.

Architecture (High Level)

- Frontend: Next.js served from S3 + CloudFront.
- API: FastAPI running in AWS Lambda behind API Gateway REST.
- Worker: dedicated Lambda for tool-heavy asynchronous runs.
- State: S3 for artifacts/memory, Upstash Redis for jobs and quotas.
- Infra as code: Terraform manages IAM, ECR, Lambda, API Gateway, S3, CloudFront, Route53, ACM.

Stack

Backend
Python, FastAPI, openai-agents, MCP

Frontend
Next.js, React, TypeScript

Cloud
AWS Lambda, API Gateway, S3, CloudFront, ECR, IAM, Route53, ACM

Data & Ops
Upstash Redis, OpenTelemetry, CloudWatch, GitHub Actions